

Trabajo Práctico: Gestión de Memoria

Parte 1: Ejercicios Prácticos

Ejercicio 1: Dado el siguiente heap representado en Python:

```
1 heap = {  
2     "A": ["B", "C"],  
3     "B": [],  
4     "C": ["D"],  
5     "D": [],  
6     "X": []  
7 }  
8 roots = ["A"]
```

- a) Dibujar el grafo de objetos.
- b) Indicar qué objetos son alcanzables.
- c) Indicar qué objetos pueden ser recolectados.

Ejercicio 2: Implementar en Python una función que devuelva el conjunto de objetos alcanzables utilizando DFS:

```
1 def alcanzables(heap, roots):  
2     pass
```

- a) Explicar el algoritmo utilizado.
- b) ¿Qué complejidad temporal y espacial tiene?

Ejercicio 3: Dado el siguiente heap con estados de marcado:

```
1 heap = {  
2     "A": {"refs": ["B"], "marked": False},  
3     "B": {"refs": ["C"], "marked": False},  
4     "C": {"refs": [], "marked": False},  
5     "X": {"refs": [], "marked": False}  
6 }  
7 roots = ["A"]
```

- a) Implementar la fase **mark**.
- b) Indicar qué objetos quedan marcados tras la ejecución.

Ejercicio 4: Continuando el ejercicio anterior, implemente la fase **sweep**:

- a) Mostrar el código de la función **sweep**.
- b) Mostrar el estado final del heap.
- c) ¿Qué objetos fueron eliminados de la memoria?

Ejercicio 5: Analice el siguiente código en C:

```
1  #include <stdlib.h>
2
3  int main() {
4      int *p = malloc(sizeof(int));
5      *p = 10;
6      return 0;
7  }
```

- a) ¿Qué problema de gestión de memoria presenta el código?
- b) ¿Cómo se solucionaría manualmente?
- c) Relacionar este escenario con el funcionamiento de un Garbage Collector automático.

Parte 2: Preguntas Teóricas

Pregunta 1: Describa detalladamente la diferencia entre memoria estática, *stack* y *heap*. ¿En cuál de ellas actúa principalmente el Garbage Collector?

Pregunta 2: ¿Qué es un *Activation Record*? Describa sus componentes principales y cómo se gestionan durante las llamadas a funciones.

Pregunta 3: Explique el concepto de **alcanzabilidad** (*reachability*). ¿Por qué es este el criterio estándar para decidir qué memoria liberar en lugar de simplemente *lo que no se usa*?

Pregunta 4: Realice una tabla comparativa entre el manejo manual de memoria (C/C++) y la recolección automática (Java/Python), indicando al menos 2 ventajas y 2 desventajas de cada uno.

Pregunta 5: Explique por qué el algoritmo de **Reference Counting** falla ante estructuras de datos cíclicas. Provea un ejemplo en pseudocódigo o Python.

Parte 3: Ejercicio Integrador (Opcional)

- ★ **Simulador de GC:** Diseñar un programa en Python que simule un recolector **Mark & Sweep**. El simulador debe permitir definir un heap, un conjunto de raíces y mostrar paso a paso cómo se identifican y eliminan los objetos *muertos*.